Report — Version 3: Generic Parser

Introduction

Version 3 introduces the **Generic Parser**.

In Version 2, the Parser worked, but it was still rigid.

It used hardcoded conditions for specific parameter combinations.

For example:

```
if parameters == ["name"]:
    ...

if parameters == ["a", "b"]:
    ...
```

This approach works with a small number of tools, but it does not scale well.

If new tools or new parameters are added, the Parser would quickly become full of `if` statements.

Version 3 solves this problem by making the Parser more generic.

Goal of Version 3

The main goal of Version 3 is to remove rigid parsing logic.

The Parser should no longer depend on specific tool configurations.

Instead, it should:

1. read the parameters required by the selected tool;
2. find the right extractor for each parameter;
3. extract each value;
4. return the final arguments dictionary.

The key idea is:

```
The Parser should not know every tool in detail.
It should know how to extract parameters.
```

What Changes from Version 2

In Version 2, the Parser was based on specific conditions.

It checked the exact list of parameters and decided what to do.

In Version 3, the Parser becomes parameter-based.

The new flow is:

```
selected tool
↓
required parameters
↓
parameter extractors
↓
arguments
```

This makes the Parser more flexible and easier to extend.

Parse Context

Version 3 introduces the concept of a parse context.

The parse context collects useful information from the user request.

Example request:

```
What is 2 + 3?
```

The context can be:

```
{
    "numbers": [2, 3],
    "name": None
}
```

This context is then reused by the parameter extractors.

The benefit is that the request is analyzed once, and the extracted information is shared across the Parser.

Parameter Extractors

Parameter extractors are small functions responsible for extracting one specific value.

Examples:

```
"name" → extract the user name
"a" → extract the first number
"b" → extract the second number
```

The mapping can look like this:

```
parameter_extractors = {
    "name": name_extractor,
    "a": first_number_extractor,
    "b": second_number_extractor
}
```

This allows the Parser to work in a generic way.

It does not need to know the complete logic for every tool.

It only needs to know which extractor belongs to each parameter.

Generic Parser Flow

Example request:

```
What is 2 + 3?
```

The Router selects:

```
"addition"
```

The tool metadata says that `addition` requires:

```
["a", "b"]
```

The Parser builds the context:

```
{
    "numbers": [2, 3],
```

```
    "name": None
}
```

Then it extracts each parameter:

```
"a" → first number → 2
"b" → second number → 3
```

The final parsed arguments are:

```
{
    "a": 2,
    "b": 3
}
```

Full Architecture in Version 3

The flow of Version 3 is:

```
User Request
↓
Router
↓
Generic Parser
↓
Executor
↓
Memory
```

The main responsibilities are now clearer:

```
Router = selects the tool
Parser = extracts arguments
Executor = validates and executes
Agent = coordinates
Memory = stores the state
```

This is already a much stronger architecture than Version 1.

Why the Generic Parser Matters

The Generic Parser is important because it makes the system easier to extend.

If a future tool requires a new parameter, the Parser should not be rewritten with more rigid conditions.

For example, if a future tool requires:

```
city
```

we can add:

```
"city": city_extractor
```

This keeps the Parser clean.

The system becomes more scalable because adding a parameter means adding an extractor, not rewriting the Parser.

Tests

The tests verify that the new Parser does not break the previous behavior.

For example:

```
agent("What is 2 + 3?")
```

should still return:

```
5
```

Another test:

```
agent("Hello, what is 2 + 3?")
```

should still select:

```
"addition"
```

And:

```
agent("Hello, my name is luca.")
```

should still return:

```
Hello Luca, nice to meet you!
```

These tests confirm that the Parser is more generic, while the agent behavior remains correct.

What Version 3 Teaches

Version 3 teaches an important architectural principle:

```
generic components scale better than rigid components
```

The Parser no longer depends on specific hardcoded cases.

It reads metadata and uses parameter extractors.

This makes the agent:

- cleaner;
- easier to extend;
- easier to read;
- easier to modify;
- closer to a professional architecture.

Limitations

Version 3 still has some limitations:

- the Router is still rule-based;
- the Parser still uses manual extraction rules;
- memory is still a simple list;
- there is no Planner;
- there is no Tool Registry;
- there is no LLM;
- the agent still performs only one action at a time.

These limitations are normal.

They will be addressed in later versions.

Conclusion

Version 3 is an important step toward a more scalable agent.

The Parser becomes generic and no longer depends on rigid `if` statements for every tool configuration.

The agent now has a cleaner flow:

```
Router
↓
Generic Parser
↓
Executor
↓
Memory
```

The next natural step is to introduce a Planner, so the agent can create a small execution plan before running a tool.